

# Guia de Estudio — Tema 09.1: Variables, Binding y Ambito

---

**Materia:** Paradigmas y Lenguajes de Programacion 2026 **Institucion:** Universidad Nacional de Tierra del Fuego — Instituto IDEI **Modulo:** Bloque post-OO, pre-tipos **Semana:** 9.1 de 15 **Tema:** 09.1 — Variables, Binding y Ambito **Duracion de la clase:** 120 min (1 clase) **Lenguaje principal:** TypeScript **Fuente bibliografica primaria:** Sebesta — *Concepts of Programming Languages* (Pearson 2019), Cap. 5 **Fuentes secundarias:** Gabbrielli & Martini (2023), Caps. 4, 5, 8; Louden & Lambert (2012), Cap. 7 **Agente:** Dra. Sofia — Study Guide Writer **Fecha:** 2026-06-28

---

## 1. Introduccion al tema

---

Cuando escribis `let x = 42` en TypeScript, parece una sola operacion simple. En realidad, acabas de disparar multiples asociaciones — entre un nombre y una direccion de memoria, entre una variable y un tipo, entre una celda y un valor — cada una en un momento distinto del ciclo de vida del programa. Esas asociaciones se llaman **bindings**, y entender cuando ocurren y que implican es el corazon de esta clase.

Este tema es la base conceptual sobre la que se apoyan closures, garbage collection y sistemas de tipos — todo lo que viene en Tema 09.2 y Tema 10. Si dominas la 5-tupla de atributos de una variable y los 6 tiempos de binding, vas a poder razonar sobre cualquier lenguaje de programacion que encuentres, no solo TypeScript.

Tambien vamos a ver por que la IA genera codigo con errores sutiles de ambito — y como prevenirlos con prompts bien disenados.

 **Filosofia de esta guia:** Si un alumno puede estudiarlo solo, lo hicimos bien. Cada seccion profundiza lo que se vio en clase; no lo repite literalmente. Los ejemplos de codigo son los mismos que se proyectaron en las filminas, con explicacion adicional.

---

## 2. Objetivos de aprendizaje

---

Al finalizar el estudio de esta guia, debes poder:

| #   | Objetivo                                                                                            | Nivel Bloom |
|-----|-----------------------------------------------------------------------------------------------------|-------------|
| OA1 | Describir la variable como 5-tupla <code>&lt;nombre, direccion, tipo, L-valor, R-valor&gt;</code>   | Recordar    |
| OA2 | Distinguir los 6 momentos de binding: diseno, implementacion, compilacion, linkeo, carga, ejecucion | Comprender  |
| OA3 | Clasificar variables segun sus 4 categorias de tiempo de vida y zona de almacenamiento              | Analizar    |
| OA4 | Comparar ambito estatico vs. dinamico: reglas de resolucion, ventajas y problemas                   | Analizar    |
| OA5 | Diferenciar tipado fuerte vs. debil y binding estatico vs. dinamico como dimensiones ortogonales    | Analizar    |
| OA6 | Aplicar el algoritmo de resolucion de ambito estatico en codigo TypeScript                          | Aplicar     |
| OA7 | Detectar errores de ambito, hoisting y variables globales silenciosas en codigo generado por IA     | Evaluar     |

**Ver filminas:** [F-00] a [F-47] — esta guia sigue el mismo orden que las filminas de la clase.

### 3. Conceptos previos necesarios

Antes de estudiar esta guia, necesitas manejar los siguientes temas de clases anteriores:

| Concepto previo                           | Tema de la cursada      | Que necesitas recordar                                                                                        |
|-------------------------------------------|-------------------------|---------------------------------------------------------------------------------------------------------------|
| Objetos y <code>this</code> en TypeScript | Tema 08 (OO TypeScript) | El <code>this</code> cambia segun como se llama la funcion; los objetos se pasan por referencia               |
| Tipos primitivos de TypeScript            | Tema 08                 | <code>number</code> , <code>string</code> , <code>boolean</code> , <code>null</code> , <code>undefined</code> |
| Funciones y parametros                    | Tema 08                 | Declaracion de funciones, paso de parametros, arrow functions                                                 |
| Estructuras de control                    | Temas previos           | <code>if</code> , <code>for</code> , bloques <code>{}</code>                                                  |
| Recursividad                              | Temas previos           | Una funcion que se llama a si misma; caso base                                                                |

**⚠ No se requiere conocimiento previo de:** aliases, closures, garbage collection, sistemas de tipos formales. Esos temas se cubren en Tema 09.2 y Tema 10. Esta guia no los desarrolla.

## 4. Desarrollo teorico

### 4.1 La variable como abstraccion de la arquitectura Von Neumann

#### El origen fisico

La arquitectura Von Neumann — la base de casi todas las computadoras modernas — tiene dos componentes fundamentales:

- **Memoria:** una coleccion de celdas numeradas. Cada celda tiene una **direccion** (su posicion fisica) y un **contenido** (el valor almacenado).
- **Procesador:** lee y escribe celdas usando instrucciones de maquina; opera siempre sobre direcciones numericas.

En lenguaje maquina, el programador trabaja con direcciones numericas como `0x7fff5b20` . Esto es propenso a errores, ilegible y dependiente del hardware. Los lenguajes de programacion introducen una capa de **abstraccion**: reemplazan las direcciones fisicas por **nombres simbolicos**.

| Elemento concreto (hardware)      | Abstraccion en el lenguaje                                                                     |
|-----------------------------------|------------------------------------------------------------------------------------------------|
| Celda de memoria fisica           | <b>Variable</b>                                                                                |
| Direccion numerica de la celda    | <b>Nombre / identificador</b> ( <code>x</code> , <code>contador</code> , <code>limite</code> ) |
| Escritura destructiva de la celda | <b>Sentencia de asignacion</b> ( <code>x = 42</code> )                                         |
| Multiples celdas contiguas        | Variable de tipo compuesto (array, struct, objeto)                                             |
| Celda de solo-lectura             | Variable constante ( <code>const</code> , <code>val</code> )                                   |

**Por que “destructiva”?** Escribir en una celda **destruye** el valor anterior — no hay historial automatico. La inmutabilidad ( `const` , `val` , `let` en Rust) es una restriccion artificial del **lenguaje**, no del hardware. El hardware siempre permite escritura destructiva.

**Ver filminas:** [F-01], [F-02]

#### Los 6 atributos de una variable

Una variable no es solo un nombre — tiene **seis atributos interdependientes**. Sebesta los formaliza en una 5-tupla extendida:

Variable = <nombre, direccion, tipo, L-valor, R-valor>

| Atributo       | Notacion | Descripcion                                                                                |
|----------------|----------|--------------------------------------------------------------------------------------------|
| Nombre         | —        | Identificador simbolico; puede no existir (variable anonima como <code>_</code> en Python) |
| Direccion      | L-value  | Celda(s) de memoria asociada(s)                                                            |
| Tipo           | —        | Rango de valores + operaciones legales + representacion binaria en memoria                 |
| Valor          | R-value  | Contenido codificado almacenado segun el tipo; puede ser indefinido                        |
| Tiempo de vida | lifetime | Periodo durante el que la variable esta vinculada a una direccion de memoria               |
| Ambito         | scope    | Rango de instrucciones donde el nombre es visible y puede ser referenciado                 |

**Definicion formal del tiempo de vida** [Sebesta, §5.4.3, Cap. 5]: “The lifetime of a variable is the time during which the variable is bound to a specific memory location. The lifetime of a variable begins when it is bound to a specific cell and ends when it is unbound from that cell.”

Cada atributo se **vincula** en un momento distinto — eso es el **binding**, que veremos en la siguiente seccion.

⚠ **Tiempo de vida ≠ ambito** — son atributos independientes. El tiempo de vida es sobre *cuando* la variable existe en memoria; el ambito es sobre *cuando* su nombre es visible. Una variable puede existir pero estar fuera de ambito.

**Ver filminas:** [F-03]

## L-value y R-value: el doble rol de una variable

Una variable puede aparecer como **destino** o como **fuentes** de datos en una asignacion:

- **L-value** (Left value): la variable aparece como destino de la asignacion. Representa la **direccion de memoria** donde se almacenara el resultado. El compilador necesita saber *donde* escribir. Ejemplo: `x` en `x = y + 1`.
- **R-value** (Right value): la variable aparece como fuente de datos. Representa el **contenido** almacenado en la celda. El compilador necesita saber *que valor* leer. Ejemplo: `y` en `x = y + 1`.

**Definicion formal** [Gabbrielli & Martini, §8.4, Cap. 8]: “l-values are those values that indicate locations and therefore are the values of expressions that can be on the left of an assignment command. On the other hand, r-values are the values that can be stored in locations.”

**Sebesta** [§5.3.2, Cap. 5]: “A variable’s value is sometimes called its r-value because it is what is required when the name of the variable appears on the right side of an assignment statement. To access the r-value, the l-value must be determined first.”

El mismo nombre puede actuar como L-value o R-value segun su posicion. Y un mismo nombre puede tener distintos L-values en distintas invocaciones (recursion) o en distintos modulos.

**Que pasa con las constantes?** `const PI = 3.14` tiene L-value (la celda donde esta almacenado el 3.14). Pero el lenguaje **prohibe usarla como L-value** en una nueva asignacion. `PI = 2.71` produce error de compilacion, no error de hardware.

**Ver filminas:** [F-04], [F-05], [F-06], [F-07]

## La 5-tupla en distintos lenguajes

**TypeScript** — tipo estatico, direccion gestionada por el runtime:

```
let x: number = 42;
// nombre: x
// tipo:  number (vinculado en compilacion)
// valor:  42 (R-value inicial)
// direccion: asignada por V8 en tiempo de carga (no visible)
// ambito: bloque donde esta declarado

const limite: number = 100;
// binding de VALOR inmutable: el lenguaje prohíbe re-asignar limite
// pero la celda de memoria sigue existiendo (tiene L-value)
```

**Python** — expone el L-value con `id()` :

```
contador = 42

# id() devuelve la direccion del objeto en el heap de Python
print(id(contador))    # ej: 140234567890 <- L-value (direccion)
print(type(contador))  # <class 'int'>      <- tipo vinculado en runtime
print(contador)        # 42                <- R-value (contenido)

# Reasignacion: Python crea un NUEVO objeto, no modifica el existente
contador = 43
print(id(contador))    # <- DIFERENTE al anterior: nueva celda, nueva direccion
# El objeto 42 sigue existiendo hasta que el GC lo recolecte

# Comparar con un objeto mutable:
lista = [1, 2, 3]
id_original = id(lista)
lista.append(4)
print(id(lista) == id_original) # True - misma direccion, contenido modificado
```

**Rust** — expone el L-value de forma segura:

```
let x = 42i32;
// tipo inferido: i32
// binding de almacenamiento: stack, liberado al salir del scope

let addr = &x as *const i32;
// addr contiene la direccion fisica de x en el stack
// esto es el L-value de x expuesto como dato del programa

// La inmutabilidad en Rust es la regla por defecto:
// let x = 42;      <- inmutable (por defecto)
// let mut y = 42;  <- mutable (requiere declaracion explicita)
```

**Go** — zero values automaticos:

```
var n int      // n = 0 (nunca hay basura, nunca hay undefined)
var s string   // s = ""
var b bool     // b = false
var p *int     // p = nil

// Go NUNCA deja una variable con valor "basura"
// Esto elimina una clase entera de bugs de C/C++
```

**Kotlin** — `var` (mutable) vs `val` (inmutable de valor):

```
var contador: Int = 42
// var -> el binding de valor puede cambiar: se puede re-asignar

val limite: Int = 100
// val -> binding de valor inmutable desde la creacion
// equivalente a TypeScript const para tipos primitivos
```

**Ver filminas:** [F-05], [F-06], [F-07]

## 4.2 Binding: el momento de la vinculacion

### Que es el binding

Un **binding** es la asociacion entre una entidad del programa (variable, operador, identificador, etc.) y uno de sus atributos (tipo, valor, direccion, significado), establecida en un **momento determinado**.

**Definicion formal** [Sebesta, §5.4, Cap. 5]: “A binding is static if it first occurs before run time begins and remains unchanged throughout program execution. If the binding first occurs during run time or can change in the course of program execution, it is called dynamic.”

El binding no es solo sobre variables — aplica a operadores, literales, subprogramas. El significado de `*` como multiplicacion es un binding establecido en tiempo de diseno del lenguaje.

### Tres preguntas clave sobre cualquier binding:

1. **Que** se esta vinculando? (el atributo — tipo, valor, direccion...)
2. **Cuando** ocurre la vinculacion? (el momento — compilacion, ejecucion...)
3. **Es permanente o puede cambiar?** (inmutable vs. reassignable)

### Por que importa el momento del binding?

- **Binding temprano** (en compilacion): mas eficiente en ejecucion, el compilador detecta errores antes de correr el programa; menos flexible para ciertos patrones.
- **Binding tardio** (en ejecucion): mas flexible, permite duck typing y estructuras genericas; costo en overhead y errores silenciosos en runtime.

El momento determina **donde se detectan los errores**: un error de tipo con binding en compilacion falla antes de ejecutar; con binding en ejecucion, falla mientras el usuario usa la app.

**Contraste con lenguajes funcionales:** En lenguajes puramente funcionales como Haskell, un valor se vincula a un nombre **una sola vez** — no existe re-asignacion. Por eso no se necesita un binding de ubicacion (L-value) separado: la variable simplemente **es** su valor, sin celda modificable.

**Ver filminas:** [F-08]

## Los 6 tiempos de binding

El binding puede ocurrir en 6 momentos distintos del ciclo de vida del programa:

| Momento                              | Que se vincula?                                   | Ejemplo concreto                                       |
|--------------------------------------|---------------------------------------------------|--------------------------------------------------------|
| <b>Diseño del lenguaje</b>           | Significados posibles para símbolos y estructuras | <code>*</code> = multiplicación (no concatenación)     |
| <b>Implementación del compilador</b> | Rango de valores de tipos primitivos              | <code>number</code> en TS = IEEE 754 float64, 64 bits  |
| <b>Compilación</b> (estático)        | Variable → tipo; operador → semántica concreta    | <code>let n: number</code> → <code>n</code> es float64 |
| <b>Linkeo</b>                        | Llamada a subprograma externo → código real       | <code>console.log</code> → función de V8 runtime       |
| <b>Carga del módulo</b>              | Variables globales/estáticas → celdas de memoria  | Variables de módulo al importar                        |
| <b>Ejecución</b> (dinámico)          | Variable → valor concreto                         | <code>n = n + 5</code> (el valor cambia en runtime)    |

## Lo que importa recordar:

- Los tres primeros momentos son **pre-ejecución**: errores detectables antes de correr.
- Linkeo y carga son **preparación del entorno de ejecución**.
- Solo el último momento ocurre **dentro del programa en marcha**.

**Refinamiento de Louden & Lambert** [§7.5, Cap. 7]: Louden distingue tres subcategorías dentro del binding estático, según cuán temprano ocurre. Del mismo modo, el binding dinámico se subdivide: puede ocurrir en la **entrada a un procedimiento** (variables locales), en la **salida** (liberación), o en cualquier instrucción de asignación durante la ejecución.

**Gabbrielli & Martini** [§4.3, Cap. 4]: “In the previous description we have ignored other important phases, such as linking and loading in which other bindings (for example for external names referring to objects in other modules).”

**Ver filminas:** [F-09], [F-10]

## Analisis de un fragmento TypeScript: 6 bindings en 2 lineas

```
let count: number; // declaracion - binding de tipo en compilacion
count = count + 5; // asignacion - binding de valor en ejecucion
```

### Analisis de bindings en el fragmento:

| Que se vincula                                        | Momento                              | Razon                                                                                     |
|-------------------------------------------------------|--------------------------------------|-------------------------------------------------------------------------------------------|
| Tipos posibles para variables                         | <b>Diseño del lenguaje</b>           | TypeScript define que tipos existen y que operaciones son validas                         |
| Tipo de <code>count</code><br>( <code>number</code> ) | <b>Compilacion</b>                   | El compilador vincula el identificador <code>count</code> con el tipo <code>number</code> |
| Rango de valores de <code>number</code>               | <b>Implementacion del compilador</b> | IEEE 754 float64: $\pm 5 \times 10^{-324}$ a $\pm 1.8 \times 10^{308}$ (64 bits)          |
| Significado del operador <code>+</code>               | <b>Compilacion</b>                   | Se resuelve la sobrecarga: <code>number + number</code> = suma numerica                   |
| Representacion del literal <code>5</code>             | <b>Implementacion del compilador</b> | El compilador decide como representar el literal <code>5</code> en bytecode               |
| Valor de <code>count</code>                           | <b>Ejecucion (runtime)</b>           | Solo cuando la linea se ejecuta, <code>count</code> toma el valor calculado               |

Ver filminas: [F-10]

## 4.3 Binding de tipos

El binding de tipos tiene **dos dimensiones ortogonales** que hay que mantener separadas:

1. **Cuando** se vincula el tipo: estatico (compilacion) vs. dinamico (ejecucion)
2. **Que tan estricto** es el chequeo: fuerte vs. debil



## Dimension 1: estatico vs. dinamico

| Caracteristica               | Binding estatico                          | Binding dinamico                                         |
|------------------------------|-------------------------------------------|----------------------------------------------------------|
| Momento de vinculacion       | Compilacion (o declaracion)               | Ejecucion — en cada asignacion                           |
| Puede cambiar el tipo?       | No — el tipo queda fijo para esa variable | Si — puede ser <code>int</code> , luego <code>str</code> |
| Deteccion de errores de tipo | En compilacion — antes de ejecutar        | En runtime — cuando el programa corre                    |
| Rendimiento                  | Alto — sin costo extra en ejecucion       | Menor — el runtime verifica tipos en cada op             |
| Flexibilidad                 | Menor — el tipo debe conocerse antes      | Mayor — util para scripting y DSLs                       |
| Ejemplos de lenguajes        | TypeScript, Kotlin, Go, Rust, Java        | Python, JavaScript, Ruby, Lua                            |

**Gabbrielli & Martini** [§8, Cap. 8]: “A language has static typing if its checking of type constraints can be conducted on the program text at compile time. Otherwise, it has dynamic typing (that is if checking happens at runtime).”

**Sebesta** [§5.4.2, Cap. 5]: “The primary advantage of dynamic binding of variables to types is that it provides more programming flexibility.”

⚠ **TypeScript es estatico en compilacion pero compila a JavaScript dinamico.** El tipado de TS desaparece en runtime — solo existe en el codigo fuente y en el compilador.

**Ver filminas:** [F-11]

## Inferencia de tipos

La inferencia es **binding estatico sin declaracion explicita** — el compilador lo deduce del contexto. No es binding dinamico — sigue ocurriendo en compilacion.

**Gabbrielli & Martini** [§8.8, Cap. 8]: “Type inference is exactly this process of the attribution of a type to an expression in which explicit type declarations of its components do not occur.”

### Dos formas de inferencia:

- **Unidireccional (flujo hacia adelante):** el tipo se deduce del valor asignado. `const items = [1, 2, 3]` → el compilador infiere `number[]`.
- **Bidireccional (contextual):** el tipo se deduce del contexto donde se usa el valor. `items.forEach(x => x.toFixed(2))` → `x` se infiere como `number` por el tipo de `items`.

```
// Inferencia simple: el tipo se deduce del valor inicial
const items = [1, 2, 3];
//   ↑ inferido: number[] (el compilador analiza el array literal)

const first = items[0];
//   ↑ inferido: number (acceso indexado a number[] devuelve number)

const total = items.reduce((acc, x) => acc + x, 0);
//   ↑ inferido: number (el acumulador inicial 0 determina el tipo)

// Inferencia bidireccional (contextual):
items.forEach(x => {
//   ↑ x: number — inferido por el tipo de items (number[])
  console.log(x.toFixed(2)); // toFixed existe en number
  // console.log(x.length); // Error: number no tiene .length
  //   el compilador detecta el error sin anotacion explicita
});
```

**Diferencia fundamental con tipado dinamico:** la inferencia es en compilacion — no hay overhead en runtime. El tipo **no cambia** durante la ejecucion una vez inferido. En tipado dinamico el tipo se determina en runtime y puede cambiar libremente.

**Ver filminas:** [F-13], [F-14]

## Binding dinamico de tipo — Python

```
# Python: binding dinamico — el tipo se determina en runtime

x = [2, 3, 4]
print(type(x))      # <class 'list'>
print(id(x))         # ej: 140234567890

# Reasignacion: x pasa a ser str — el tipo cambio completamente
x = "uno, dos, tres"
print(type(x))      # <class 'str'>
print(id(x))         # <- DIFERENTE: nuevo objeto en heap

# Python ES fuertemente tipado: no permite operaciones incompatibles sin conversion
x + 42               # TypeError: can only concatenate str (not "int") to str
                    # el error aparece en runtime, no en compilacion

# Para mezclar, se requiere conversion explicita:
x + str(42)          # -> "uno, dos, tres42"
int("5") + 42        # -> 47
```

**Ver filminas:** [F-15]

## Dimension 2: fuerte vs. debil

Esta dimension es **ortogonal** a estatico/dinamico. Un lenguaje puede ser cualquier combinacion de las cuatro.

| Característica                    | Tipado fuerte                                        | Tipado debil                                            |
|-----------------------------------|------------------------------------------------------|---------------------------------------------------------|
| <b>Conversiones implícitas</b>    | No — o solo conversiones “seguras” y predecibles     | Si — coerciones arbitrarias en cualquier operacion      |
| <b>Comportamiento ante mezcla</b> | Error explicito (compilacion o excepcion en runtime) | Resultado silencioso posiblemente incorrecto            |
| <b>Trazabilidad de bugs</b>       | Alta — el error aparece cerca de la causa real       | Baja — el error aparece lejos de donde esta el problema |
| <b>Ejemplos de lenguajes</b>      | Haskell, Rust, Python, TypeScript strict             | C (muchas coerciones), JavaScript, PHP                  |

**Gabbrielli & Martini** [§8.3, Cap. 8]: “Languages with strong type checking tend to have few coercions. On the other hand, in a language like C, the type system is designed to be by-passed and so permits numerous coercions (from characters to integers, from long reals to short, from long integers to short).”

**Sebesta** [§5.4.2, Cap. 5]: “The value of strong typing is weakened by coercion. Languages with a great deal of coercion, like C, and C++, are less reliable than those with no coercion, such as ML and F#.”

Una **coercion** es una conversion de tipo insertada automaticamente por el compilador o runtime cuando detecta compatibilidad entre tipos. En el tipado debil, estas coerciones ocurren silenciosamente en cualquier operacion, produciendo resultados incorrectos sin ningun aviso de error.

#### Las cuatro combinaciones:

|                 | Fuerte                           | Debil           |
|-----------------|----------------------------------|-----------------|
| <b>Estatico</b> | Haskell, Rust, TypeScript strict | C, C++          |
| <b>Dinamico</b> | Python                           | JavaScript, PHP |

**Ver filminas:** [F-12]

## Coerciones en JavaScript (tipado debil)

```
// JavaScript realiza coerciones implícitas sin avisar

// + con string: el número se convierte a string (concatenación)
"5" + 3    // -> "53"    (number 3 -> string "3" -> concatenación)
"5" + true // -> "5true"

// - siempre es aritmético: el string se convierte a number
"5" - 3    // -> 2        (string "5" -> number 5 -> resta)
"5" * 2    // -> 10

// == hace coerciones, === no
0 == false // -> true     (false se convierte a 0)
0 === false // -> false   (sin coerción, tipos diferentes)
"" == false // -> true

null == undefined // -> true   (caso especial del estándar)
null === undefined // -> false

// Los casos más famosos:
[] + []      // -> ""
[] + {}      // -> "[object Object]"
{} + []      // -> 0   (en algunos contextos de evaluación)
```

## TypeScript strict y Python rechazan las coerciones

```
// TypeScript strict – error de compilación al mezclar tipos
const a: string = "5";
const b: number = 3;

a + b;
// Error TS2365: Operator '+' cannot be applied to types 'string' and 'number'
// El error aparece en el editor, antes de ejecutar cualquier código

// Si queremos concatenar, la conversión debe ser explícita:
a + b.toString(); // -> "53" (conversión explícita, intención clara)
String(b) + a;     // -> "35"
```

```
# Python (dinámico + fuerte) – excepción explícita en runtime
"5" + 3
# TypeError: can only concatenate str (not "int") to str
# Python no silencia el error – lo convierte en excepción inmediata

# La conversión debe ser explícita (igual que TypeScript):
"5" + str(3) # -> "53"
int("5") + 3 # -> 8
```

**Ver filminas:** [F-16], [F-17]

## 4.4 Binding de almacenamiento: las 4 categorías de variables

El **tiempo de vida** de una variable es el periodo durante el cual esta vinculada a una dirección específica. Según cuando ocurre el binding de almacenamiento y donde se guarda la variable, existen cuatro categorías.

**Sebesta** [§5.4.3, Cap. 5]: “The memory cell to which a variable is bound must be taken from a pool of available memory. This process is called allocation. Deallocation is the process of placing a memory cell that has been unbound from a variable back into the pool of available memory.”

| Categoría           | Cuando se vincula la dirección?                         | Cuando se libera?                  | Zona de memoria   | Permite recursión? |
|---------------------|---------------------------------------------------------|------------------------------------|-------------------|--------------------|
| 1 — Estáticas       | Antes de ejecutar el programa                           | Al terminar el programa            | Segmento estático | No                 |
| 2 — Stack-dynamic   | Al elaborar la declaración (en llamada)                 | Al retornar del subprograma        | Pila de llamadas  | Si                 |
| 3 — Heap explícitas | Al ejecutar <code>new</code> / <code>Box : : new</code> | Al ejecutar GC / <code>drop</code> | Heap              | Si                 |
| 4 — Heap implícitas | En cualquier sentencia de asignación                    | Al ejecutar GC                     | Heap              | Si                 |

**Por que las estáticas no permiten recursión efectiva?** En Cat. 1, todas las llamadas a la misma función **comparten la misma celda de memoria**. Si una función se llama a si misma, la segunda llamada sobrescribe los valores de la primera. Las invocaciones no pueden coexistir con variables independientes — no hay frames separados.

**Ver filminas:** [F-18]

## Categoría 1 — Variables estáticas

Las variables estáticas existen desde el inicio del programa hasta el final. Su dirección de memoria es **conocida en tiempo de compilación**.

### Características:

- La celda se reserva **antes de que el programa empiece** y no se libera hasta que termina.
- No hay overhead de `allocate/deallocate` en cada llamada a función.
- Persisten entre llamadas (historia) — útil para caches, contadores globales.

### Cuando usar variables estáticas:

- Constantes del módulo: valores que no cambian durante toda la ejecución.
- Configuración: parámetros cargados al inicio que aplican a todo el módulo.
- Cache / estado compartido: valores que deben persistir entre llamadas a la misma función.

**La restricción de recursión:** En C, FORTRAN y otros lenguajes con asignación estática para locales, la función no puede llamarse a si misma de forma útil. Cada llamada trabaja con **la misma celda** de memoria para sus variables locales. FORTRAN 77 prohíbe la recursión directamente por esta razón — cada subprograma tiene una única área de datos, fija en memoria, que se reutiliza en cada invocación.

**En lenguajes modernos:** TypeScript, Python, Java, Go y Kotlin usan stack-dynamic para las variables locales. La estática existe para variables de módulo, constantes globales y `companion object` en Kotlin.

```
// Variables de modulo: binding de almacenamiento antes de ejecucion
const VERSION = "1.0.0";      // constante estatica – direccion fija desde carga
let sesionesActivas = 0;      // variable estatica mutable – existe todo el tiempo de vida

// Patron de inicializacion lazy (una sola vez en toda la vida del modulo):
let _cache: Map<string, string> | null = null;

export function getCache(): Map<string, string> {
  // _cache se inicializa la primera vez que se llama a getCache()
  // Las llamadas siguientes reutilizan la misma instancia (misma celda)
  _cache ??= new Map();
  return _cache;
}

// Traza de ejecucion:
// 1ra llamada: _cache === null -> se crea el Map -> _cache apunta al Map
// 2da+ llamada: _cache !== null -> se devuelve el mismo Map
```

```
class Sesion private constructor(val id: Int) {

  companion object {
    // companion object = espacio estatico de la clase Sesion
    // Sus propiedades y funciones existen desde que la clase se carga

    private var contadorGlobal = 0 // estatica – un binding por toda la JVM

    fun nueva(): Sesion {
      contadorGlobal++ // modifica la celda compartida por todos
      return Sesion(contadorGlobal)
    }

    fun totalSesiones(): Int = contadorGlobal
  }
}

// Uso:
val s1 = Sesion.nueva() // id = 1, contadorGlobal = 1
val s2 = Sesion.nueva() // id = 2, contadorGlobal = 2
println(Sesion.totalSesiones()) // 2 – contadorGlobal es la misma celda para todos
```

**Ver filminas:** [F-19], [F-20], [F-21]

## Categoria 2 — Variables dinamicas de pila (Stack-dynamic)

Creadas al activar el subprograma, destruidas al retornar. Son las variables mas comunes en TypeScript: cualquier `let` o `const` declarado dentro de una funcion es stack-dynamic.

**Sebesta** [§5.4.3.2, Cap. 5]: “Stack-dynamic variables are those whose storage bindings are created when their declaration statements are elaborated, but whose types are statically bound. Elaboration of such a declaration refers to the storage allocation and binding process indicated by the declaration, which takes place when execution reaches the code to which the declaration is attached — during run time.”

El concepto de **elaboracion** es clave: la declaracion `let resultado = 0` no reserva memoria en compilacion. La reserva se produce cuando la **ejecucion llega** a esa declaracion. Para una variable local dentro de una funcion, la elaboracion ocurre en cada llamada a esa funcion, creando un binding fresco en el frame correspondiente.

**Por que son la norma en lenguajes modernos:**

- Sin gestion manual de memoria: el sistema controla el ciclo de vida.
- Sin basura: la celda se libera exactamente cuando ya no se necesita.
- Sin interferencia entre llamadas: cada llamada tiene sus propias celdas.

### La pila de llamadas (call stack):

- Estructura LIFO (Last In, First Out): la ultima funcion en llamarse es la primera en retornar.
- Cada llamada agrega un **frame** (registro de activacion) con las variables locales.
- Los frames no se comparten — cada invocacion tiene los suyos propios.

**Por que permiten recursion?** Cada llamada recursiva genera un **frame nuevo** en la pila. Las variables de diferentes invocaciones coexisten en frames distintos, sin interferencia. No importa que sea la misma funcion — si hay frame propio, hay variables propias.

```
function calcular(n: number): number {
  // Al entrar a calcular(): se reservan dos celdas en la pila
  let resultado = 0; // celda propia de ESTA llamada
  let temp = n * 2; // celda propia de ESTA llamada

  console.log(`temp = ${temp}`);
  return resultado + temp;
  // Al retornar: las celdas de resultado y temp son DESTRUIDAS
  // Si se llama calcular() otra vez, se crean nuevas celdas (nuevas direcciones)
}

// Estas dos llamadas son completamente independientes:
calcular(5); // crea sus propias celdas: resultado=0, temp=10
calcular(7); // crea sus propias celdas: resultado=0, temp=14

// No hay interferencia entre las dos invocaciones:
// la temp de calcular(5) y la temp de calcular(7) son celdas diferentes
```

### Factorial recursivo — el ejemplo canonico:

```
function factorial(n: number): number {
  // CADA llamada a factorial() tiene su propia celda para n
  // La n de factorial(3) y la n de factorial(2) son celdas distintas

  if (n <= 1) return 1; // caso base: retorna, libera el frame

  return n * factorial(n - 1);
  //   ↑ usa la n de ESTE frame
  //   ↑ crea un NUEVO frame para n-1
}

console.log(factorial(3)); // 6

// Traza de ejecucion:
// factorial(3): espera a factorial(2)
// factorial(2): espera a factorial(1)
// factorial(1): n <= 1 -> retorna 1 (sin llamada recursiva mas)
// factorial(2): recibe 1, calcula 2*1=2, retorna 2
// factorial(3): recibe 2, calcula 3*2=6, retorna 6
```

⚠ Los detalles internos del activation record (static link, dynamic link, direccion de retorno) pertenecen a Tema 13 (Abstraccion Procedural). En esta clase solo interesa la consecuencia: **frame independiente** → **recursion segura**.

**Ver filminas:** [F-22], [F-23], [F-24]

## Categoría 3 — Variables dinámicas de Heap explícitas

Asignadas y liberadas de forma controlada (manual o por GC/ownership). El programador solicita la memoria explícitamente ( `new` , `Box::new` , `malloc` ).

**Sebesta** [§5.4.3.3, Cap. 5]: “Explicit heap-dynamic variables are nameless (abstract) memory cells that are allocated and deallocated by explicit run-time instructions written by the programmer. These variables, which are allocated from and deallocated to the heap, can only be referenced through pointer or reference variables.”

La característica distintiva es que **no tienen nombre propio** — se acceden siempre a través de un puntero o referencia. En TypeScript se accede a objetos del heap con variables que almacenan referencias; el objeto en sí es anónimo.

### Liberación:

- **Automatica (GC):** Java, TypeScript, Python, Go → el runtime detecta cuando no hay referencias.
- **Manual:** C → `free()` explícito — riesgo de memory leaks y use-after-free.
- **Ownership:** Rust → el compilador determina cuando liberar, sin GC y sin errores.

```
// TypeScript: new crea el objeto en el heap; el GC lo libera automáticamente
class Nodo {
  constructor(
    public valor: number,
    public siguiente: Nodo | null = null
  ) {}
}

let cabeza = new Nodo(42);
// cabeza -> Nodo(42) en el heap (nueva direccion)

cabeza = new Nodo(99);
// cabeza -> Nodo(99) en el heap (nueva direccion)
// Nodo(42) ya no tiene referencias -> el GC lo recolectará
// El programador no necesita liberar memoria manualmente
```

```
// Rust: drop() automático al salir del scope — sin GC, determinístico
fn main() {
  let elemento = Box::new(42i32);
  // elemento -> valor 42 en el heap (Box gestiona la direccion)

  println!("{}", elemento); // 42
} // fin del scope -> destructor de Box se llama automáticamente
// la memoria se libera AQUÍ, sin GC, sin memory leak posible
```

**Rust: ownership como alternativa al GC** [Gabbrielli & Martini, §14, Cap. 14]: “Each value in Rust is attached to a variable, which is its exclusive ‘owner’. Owners can transfer the ownership of a value to other variables, and other variables can borrow values from their owners.” El compilador de Rust verifica en compilación que cada valor tenga exactamente un dueño y que los préstamos temporales ( `&T` ) no superen la vida del valor original.

**Ver películas:** [F-25], [F-26]



Categoria 4 — Variables dinamicas de Heap implicitas

Todos sus atributos (tipo, valor, direccion) se establecen cuando se les asigna un valor. El binding de almacenamiento ocurre en cada asignacion, sin que el programador lo solicite.

**Sebesta** [§5.4.3.4, Cap. 5]: “Implicit heap-dynamic variables are bound to heap storage only when they are assigned values. In fact, all their attributes are bound every time they are assigned.”

```
# Python: heap implicita – cada asignacion puede cambiar todos los atributos

x = [1, 2, 3]
print(type(x), id(x))      # <class 'list'> 140234567890

x = "uno, dos, tres"
print(type(x), id(x))      # <class 'str'> 140234567999  <- todo cambio
# ↑ tipo cambio: list -> str
# ↑ direccion cambio: nueva celda en el heap
# ↑ valor cambio: [1,2,3] -> "uno, dos, tres"

x = 42
print(type(x), id(x))      # <class 'int'> 140234568100

# El objeto [1,2,3] anterior quedo sin referencias
# El GC de Python lo recolecta eventualmente

# Comparar con Cat. 1 o Cat. 2:
# En Cat. 1/2 el tipo es fijo y la direccion no cambia durante la vida de la variable
```

```
// TypeScript con 'any' – se aproxima a Cat. 4 (desaconsejado)
let x: any = [1, 2, 3];
x = "uno, dos, tres"; // binding de tipo cambia – TypeScript permite con any
// Con tipos declarados: TypeScript es Cat. 2/3 (tipo fijo en compilacion)
```

Trade-offs entre heap explicita e implicita:

|                         | Heap explicita        | Heap implicita |
|-------------------------|-----------------------|----------------|
| Control del programador | Alto                  | Bajo           |
| Riesgo de error         | Memory leaks (sin GC) | Menor          |
| Flexibilidad del tipo   | Moderada              | Maxima         |

Ver filminas: [F-25], [F-27]

4.5 Ambito (Scope)

Definicion

El **ambito** de una variable es el rango de instrucciones del programa donde el nombre de esa variable es **visible y puede ser usado**.

- Un nombre es “visible” si puede aparecer en una expresion sin causar error de nombre no definido.
- El ambito **estatico** (o **lexico**) usa la estructura del codigo fuente para determinar la visibilidad.
- Es el modelo de todos los lenguajes modernos: TypeScript, Python, Java, Go, Rust.

⚠ **Ambito ≠ tiempo de vida** — la visibilidad del nombre es independiente de la existencia en memoria. Una variable puede existir (estar en memoria) pero estar fuera de ambito (nombre invisible).

## Ambito estatico (lexico)

Introducido por ALGOL 60. Determinado en **tiempo de compilacion**.

### Algoritmo de resolucion:

1. Buscar el nombre en el **ambito local** (el bloque `{}` actual).
2. Si no se encuentra → subir al **bloque padre estatico** (el bloque que lo contiene en el codigo).
3. Continuar hacia afuera, nivel por nivel, hasta el **ambito global del modulo**.
4. Si no se encuentra en ningun nivel → **error de compilacion** (nombre no declarado).

El ambito comienza en la declaracion:

```
console.log(x); // Error: 'x' used before its declaration
let x = 10;      // <- el ambito de x comienza aqui, no al principio del bloque
```

Esta politica es una de las razones por las que TypeScript recomienda declarar variables cerca de donde se usan.

```
let x = 10; // ambito: modulo (nivel 0)

function externa() {
  let y = 20; // ambito: funcion externa (nivel 1)

  function interna() {
    let z = 30; // ambito: funcion interna (nivel 2)

    // Resolucion de cada nombre (algoritmo de busqueda hacia afuera):
    console.log(x); // 1. ¿x en nivel 2? No
                    // 2. ¿x en nivel 1? No
                    // 3. ¿x en nivel 0? Si -> x = 10

    console.log(y); // 1. ¿y en nivel 2? No
                    // 2. ¿y en nivel 1? Si -> y = 20

    console.log(z); // 1. ¿z en nivel 2? Si -> z = 30
  }

  // console.log(z); // z no existe en nivel 1 ni arriba – error de compilacion
}

// console.log(y); // y no existe en nivel 0 – error de compilacion
```

**Gabbrielli & Martini** [§4.3, Cap. 4]: “A variable becomes visible at the declaration, but the storage binding (and initialization, if it is specified in the declaration) occurs when the function or method begins execution.”

Ver filminas: [F-28], [F-29]

## Ambito dinamico

Determinado en **tiempo de ejecucion** segun la cadena de llamadas.

**Gabbrielli & Martini** [§4.3, Cap. 4]: “According to the rule of dynamic scope, the valid association for a name  $X$ , at any point  $P$  of a program, is the most recent (in the temporal sense) association created for  $X$  which is still active when the control flow arrives at  $P$ .”

La implementacion es conceptualmente sencilla: para resolver una referencia no local al nombre  $x$ , basta con recorrer la pila de activacion hacia atras, desde el frame mas reciente, hasta encontrar un frame que declare  $x$ . El primer binding activo encontrado es el que se usa.

**Gabbrielli & Martini** [§5, Cap. 5]: “Conceptually, the implementation of the dynamic scope rule is much simpler than the one for static scope.”

**Problemas del ambito dinamico** [Sebesta, §5.5.4, Cap. 5]:

1. **Imposibilidad de verificacion estatica de tipos:** las referencias a variables no locales no pueden verificarse en compilacion, porque que variable “corresponde” depende del flujo de ejecucion.
2. **Codigo dificil de leer:** para entender que valor tiene una variable en un punto dado hay que reconstruir mentalmente la cadena de llamadas activas en ese momento — informacion que no esta en el texto del programa.
3. **Accesos mas costosos:** resolver una referencia no local requiere recorrer la pila en runtime; el acceso a no-locales en ambito estatico se compila a una sola direccion por nivel estatico.

| Caracteristica                | Ambito estatico (lexico)                 | Ambito dinamico                                               |
|-------------------------------|------------------------------------------|---------------------------------------------------------------|
| Momento de resolucion         | Compilacion                              | Ejecucion                                                     |
| Que determina la visibilidad? | Estructura lexica del codigo fuente      | Cadena de llamadas activas en ese instante                    |
| Verificable en compilacion?   | Si — el compilador valida todos los usos | No — depende del flujo de ejecucion                           |
| Legibilidad                   | Alta — la visibilidad se ve en el codigo | Baja — hay que rastrear quien llamo a quien                   |
| Lenguajes                     | TypeScript, Python, Go, Rust, Java       | Emacs Lisp (original), Perl <code>local</code> , shells POSIX |

**Nota historica:** Ambito estatico introducido por **ALGOL 60**. Los primeros dialectos de **Lisp** usaban ambito dinamico; **Common Lisp** (1984) adopto estatico como default.

Ver filminas: [F-30]

## this en JavaScript: semantica dinamica

El valor de `this` en una funcion regular depende de **como** se llama la funcion, no de donde esta escrita. Las arrow functions capturan `this` lexicamente.

```
class Timer {
  delay = 1000;

  // Funcion regular: this es DINAMICO – depende del contexto de llamada
  startConFunctionRegular() {
    setTimeout(function() {
      // this NO es la instancia Timer – lo perdio setTimeout
      // En strict mode: this === undefined

      console.log(this?.delay); // undefined
    }, this.delay);
  }

  // Arrow function: this es LEXICO – capturado en compilacion
  startConArrow() {
    setTimeout(() => {
      // this ES la instancia Timer – garantizado por el ambito lexico

      // El compilador sabe en compilacion que es this dentro de la arrow
      console.log(this.delay); // 1000
    }, this.delay);
  }
}

const t = new Timer();
t.startConFunctionRegular(); // -> undefined (this dinamico, perdido en setTimeout)
t.startConArrow();          // -> 1000 (this lexico, capturado en startConArrow)
```

Este es el ejemplo mas concreto de como el ambito dinamico causa bugs y como la arrow function es la solucion lexica.

Ver filminas: [F-31]

## Agujeros de ambito (Scope holes) — shadowing

El **shadowing** ocurre cuando una declaracion en un bloque interno usa el **mismo nombre** que una variable ya existente en un bloque exterior. La variable interior **“oculta”** a la exterior dentro de su bloque. La variable exterior cae en un **“scope hole”**: sigue existiendo, pero no es accesible dentro del bloque interno.

**El mecanismo:** el algoritmo de resolucion de ambito **siempre prefiere** el binding mas local. Si encuentra el nombre en el bloque actual, **no continua buscando** en los padres.

```

let x = 10; // x del modulo (nivel 0)

function procesarLista(items: number[]): void {

    for (const item of items) {
        const x = item * 2;
        //   ↑ x LOCAL al bloque del for (nivel 2)
        //   <- SHADOW: oculta al x del modulo dentro de este bloque

        // "scope hole" de x exterior: comienza aqui
        console.log(x); // usa x LOCAL: 20, 40, 60, ...
    }
    // Aqui termina el scope hole

    console.log(x); // x del modulo nuevamente visible: 10
}

procesarLista([10, 20, 30]);
// Salida:
//   20
//   40
//   60
//   10 <- x del modulo, nunca fue modificado

// ESLint con @typescript-eslint/no-shadow reportaria:
//   warning: 'x' is already declared in the upper scope (no-shadow)

```

**Por que es problematico:** el codigo parece estar usando la variable exterior pero en realidad usa la interior. Los linters alertan sobre shadowing porque es una fuente frecuente de bugs sutiles. TypeScript por si solo no bloquea el shadowing — requiere configuracion explicita de ESLint.

**Ver filminas:** [F-32], [F-33]

## 4.6 Entorno de referencia, constantes e inicializacion

### Entorno de referencia

El **entorno de referencia** es la “foto” de todos los nombres visibles en un punto del programa.

**Sebesta** [§5.5, Cap. 5]: “The referencing environment of a statement is the collection of all variables that are visible in the statement. The referencing environment of a statement in a static-scoped language is the variables declared in its local scope plus the collection of all variables of its ancestor scopes that are visible.”

En un lenguaje con ambito estatico, el entorno de referencia de una sentencia es necesario mientras esa sentencia esta siendo compilada — el compilador construye las estructuras de codigo y datos que permiten las referencias a variables de otros scopes durante la ejecucion. No se necesita calcular nada en runtime para saber que nombres son visibles.

**Variacion a lo largo del programa:** el entorno de referencia **no es fijo** — cambia cada vez que se entra o sale de un bloque. Al entrar a una funcion: se agrega su scope local al entorno. Al salir: ese scope se elimina.

**Componentes del entorno en TypeScript:**

- Variables locales del bloque actual
- Parametros de la funcion actual
- Variables de funciones externas (capturadas por closure)
- Variables del modulo (top-level)
- Identificadores globales del runtime ( `console` , `Math` , `undefined` , etc.)

**Relacion con closures:** una **closure** “congela” el entorno de referencia en el momento en que se crea la funcion. El entorno capturado puede incluir variables que ya salieron del stack pero siguen vivas por la closure. Esto se estudia en detalle en **Tema 09.2**.

**Ver filminas:** [F-34]

## Constantes

Una **constante** es un identificador cuyo valor se fija en un momento determinado y el lenguaje **prohíbe modificarlo** despues.

**Cuando se fija el binding de valor:**

- **En compilacion / declaracion:** `const PI = 3.14159` — el valor es parte del codigo fuente.
- **En tiempo de carga:** `const VERSION = pkg.version` — se determina al iniciar el modulo.
- **En ejecucion (primera asignacion):** `val limite = calcularLimite()` en Kotlin — inmutable desde el momento de la asignacion.

**`const` en TypeScript: la referencia, no necesariamente el objeto:**

- Para **tipos primitivos** ( `number` , `string` , `boolean` ): `const` hace inmutable el valor.
- Para **objetos y arrays:** `const` hace inmutable la **referencia** — el objeto interno puede mutar.
- Para inmutabilidad profunda se necesita `Object.freeze()` o tipos `readonly` en TypeScript.

```
// Caso 1: tipo primitivo – const hace inmutable el VALOR
const PI = 3.14159;
// PI = 2.71; // Error: Assignment to constant variable

// Caso 2: objeto – const hace inmutable la REFERENCIA (direccion)
const CONFIG = { debug: false, maxRetries: 3 };
CONFIG.debug = true; // valido: el objeto es mutable, solo la referencia es const
CONFIG.maxRetries = 5; // valido

// CONFIG = { debug: true }; // Error: Cannot assign to 'CONFIG'

// Caso 3: inmutabilidad profunda con Object.freeze
const FROZEN = Object.freeze({ debug: false });
FROZEN.debug = true;
// En JavaScript: silencioso (el cambio se ignora sin error)
// En TypeScript strict: Error de tipo – readonly property

// Patron recomendado para objetos de configuracion inmutables:
const SETTINGS = Object.freeze({
  timeout: 5000,
  retries: 3,
} as const);
// 'as const' asegura que TypeScript trate todos los campos como literales de tipo
```

**Por que las constantes importan en binding:** representan un **binding de R-value que no puede reasignarse**. El L-value (direccion) sigue existiendo — la celda esta ocupada durante toda la vida del modulo. El compilador puede optimizar accesos a constantes porque su valor es conocido en compilacion.

**Ver filminas:** [F-35], [F-36]

Inicializacion

**Sebesta** [§5.4.3, Cap. 5]: “In many instances, it is convenient for variables to have values before the code of the program or subprogram in which they are declared begins executing. The binding of a variable to a value at the time it is bound to storage is called initialization.”

**Sebesta** [§5.4.3, Cap. 5]: “The discussion of binding values to named constants naturally leads to the topic of initialization, because binding a value to a named constant is the same process, except it is permanent.”

Comparativa de lenguajes — que pasa si se usa una variable antes de inicializarla:

| Lenguaje          | Variable no inicializada                                                             | Momento de deteccion           |
|-------------------|--------------------------------------------------------------------------------------|--------------------------------|
| C                 | Estaticas → 0 automatico; locales → <b>basura</b> (contenido previo de la celda)     | Ejecucion (undefined behavior) |
| C++               | Igual que C para primitivos; constructores para objetos                              | Ejecucion                      |
| Java              | Campos → valores por defecto (0, false, null); locales → error de compilacion        | Variables locales: compilacion |
| TypeScript strict | <b>Error de compilacion</b> — analisis de flujo detecta el camino sin inicializacion | Compilacion                    |
| Python            | <code>NameError</code> — el interprete lanza excepcion al acceder                    | Runtime                        |
| Go                | <b>Zero values automaticos:</b> 0, false, <code>""</code> , nil — nunca hay basura   | Siempre seguro                 |
| Rust              | Error de compilacion — el compilador exige inicializacion antes del primer uso       | Compilacion                    |

```
let n: number;
console.log(n); // Error TS2454: Variable 'n' used before being assigned
// El compilador analiza el flujo y detecta que n puede no estar inicializada
```

**Ver filminas:** [F-37], [F-38]

## 4.7 Bloque IA — Errores de ambito en codigo generado por IA

### Por que la IA comete errores de scope

Los modelos de lenguaje aprenden de corpus de codigo extraido de la web. El corpus incluye **millones de archivos pre-ES6** con `var`, variables globales y shadowing silencioso. En JavaScript no-strict, muchos de estos errores **no generan excepciones** — el codigo “funciona” aunque sea incorrecto. El modelo aprende que estos patrones son validos porque el corpus los usa sin marcarlos como error.

### Tres patrones concretos a reconocer:

1. **var hoisting**: variable se “eleva” al inicio de la funcion — da `undefined` en lugar de `ReferenceError`.
2. **Variable global silenciosa**: dependencias ocultas no declaradas como parametro.
3. **Shadowing inesperado**: a que variable se refiere realmente el codigo generado?

Ver filminas: [F-39]

### Patron 1: `var` hoisting silencioso

En JavaScript/TypeScript, las declaraciones `var` son **elevadas** (*hoisted*) al inicio de la funcion. El compilador mueve la declaracion (no la inicializacion) al principio del scope de funcion. Esto significa que la variable **existe** desde el inicio de la funcion, aunque su valor sea `undefined`.

**Temporal Dead Zone (TDZ)**: con `let / const`, la variable existe en el scope pero **no puede usarse** hasta que se alcanza su declaracion. Usar la variable antes de su declaracion lanza `ReferenceError` — un error explicito y localizable. Con `var` no hay TDZ: cualquier uso antes de la asignacion da `undefined` en silencio.

|                              | <code>var</code>                                         | <code>let y const</code>                                       |
|------------------------------|----------------------------------------------------------|----------------------------------------------------------------|
| <b>Ambito</b>                | Funcion (no respeta bloques <code>{ }</code> )           | Bloque ( <code>{ }</code> )                                    |
| <b>Hoisting</b>              | Si — sube al inicio de la funcion                        | Si — pero entra en Temporal Dead Zone                          |
| <b>Temporal Dead Zone</b>    | No hay TDZ — da <code>undefined</code> antes de asignar  | Hay TDZ — da <code>ReferenceError</code> antes de declarar     |
| <b>Uso antes de declarar</b> | <code>undefined</code> (silencioso, dificil de rastrear) | <code>ReferenceError</code> (explicito, facil de diagnosticar) |



```
// Codigo tipico de IA entrenada en corpus pre-ES6:
function procesar(activo: boolean) {
  if (activo) {
    var resultado = "ok"; // <- var: scope de FUNCION, no de bloque
  }
  // "var resultado" fue elevada al inicio de procesar():
  // equivale a:
  //   var resultado;           // undefined

  //   if (activo) { resultado = "ok"; }

  console.log(resultado); // undefined si activo = false — NO lanza error
  // El error silencioso puede ocurrir lejos de la causa real
}

// Con let: error explicito y localizable
function procesar2(activo: boolean) {
  if (activo) {
    let resultado = "ok"; // <- let: scope del BLOQUE if
  }
  // console.log(resultado); // ReferenceError: resultado is not defined
  // El error aparece exactamente donde esta el problema
}
```

**Ver filminas:** [F-40], [F-41]

## Patron 2: Variable global silenciosa

Una **funcion pura** depende solo de sus parametros y produce un resultado sin modificar el entorno externo. Cuando una funcion accede a una variable declarada **fuera de su firma**, crea una **dependencia implicita**.

### Por que es problematico:

- La firma de la funcion no refleja todas sus dependencias — el lector no puede entender que hace sin leer el cuerpo.
- El resultado de la funcion puede cambiar segun el **orden de las llamadas anteriores** — no es predecible desde sus parametros.
- Las pruebas unitarias se complican: hay que inicializar el estado global antes de cada test.
- Los efectos secundarios son invisibles para el compilador — no hay error de compilacion.

```
// Codigo con efecto secundario implicito (generado por IA):
let total = 0; // variable global oculta — no aparece en la firma

function acumular(n: number) {
  total += n; // <- modifica estado externo sin declararlo en la firma
  return total; // <- el resultado depende de todas las llamadas anteriores
}

acumular(5); // total = 5
acumular(3); // total = 8
acumular(5); // total = 13 (mismo argumento, resultado diferente)

// Correcto — todas las dependencias son parametros explicitos:
function acumularPuro(total: number, n: number): number {
  return total + n; // mismos argumentos -> siempre el mismo resultado
}

const r1 = acumularPuro(0, 5); // -> 5
const r2 = acumularPuro(r1, 3); // -> 8
const r3 = acumularPuro(r2, 5); // -> 13
// Predecible, testeable de forma aislada, sin estado oculto
```

Ver filminas: [F-42], [F-43]

### Patron 3: Shadowing inesperado

```
const limite = 100; // limite del modulo (nivel 0)

// Codigo generado por IA con shadowing inesperado:
function validar(items: number[]) {
  const limite = items.length; // <- SHADOW: nueva variable con el mismo nombre
  //                          oculta al limite = 100 del modulo

  return items.filter(x => x < limite);
  //                ↑ que limite? -> el LOCAL (items.length), NO el 100
  // Si items = [50, 120, 80] y items.length = 3:
  //   items.filter(x => x < 3) -> [] <- vacio (ningun elemento < 3)
  //   El comportamiento correcto seria filtrar los que son < 100
}

// Correcto – sin shadowing, intencion explicita:
function validarSinShadow(items: number[], valorLimite: number): number[] {
  const cantidadItems = items.length; // nombre diferente – sin shadow
  return items.filter(x => x < valorLimite); // sin ambigüedad
}
```

Ver filminas: [F-44]

### Prompt seguro para variables en TypeScript

Instrucciones explicitas al modelo producen codigo con buenas practicas de scope:

```
"TypeScript strict mode.
Declara todo con let/const (nunca var).
Sin variables globales – todas las dependencias
son parametros explicitos con sus tipos declarados.
Declara el tipo de cada parametro y el tipo de retorno."
```

### Por que funciona cada restriccion:

- **strict mode**: activa deteccion de variables usadas antes de asignacion (analisis de flujo de tipos).
- **let/const (nunca var)**: elimina el hoisting problematico — cualquier uso fuera de scope da `ReferenceError`.
- **Parametros explicitos**: el modelo entiende que no puede usar variables de fuera de la funcion.
- **Tipos en firma**: el compilador puede verificar la correccion antes de ejecutar.

Ver filminas: [F-46]

## 5. Ejemplos trabajados paso a paso

### Ejemplo 1: Analisis de los 6 tiempos de binding en un fragmento TypeScript

**Enunciado:** Dado el siguiente fragmento, identificar que binding ocurre en cada momento.

```
let count: number;  
count = count + 5;
```

## Resolucion paso a paso:

### Paso 1 — Identificar todas las entidades y atributos involucrados:

- La variable `count` (entidad) con su tipo (atributo)
- La variable `count` con su valor (atributo)
- El operador `+` (entidad) con su significado (atributo)
- El literal `5` (entidad) con su representacion (atributo)
- El tipo `number` (entidad) con su rango de valores (atributo)

### Paso 2 — Clasificar cada binding por momento:

| Binding                                               | Momento                              | Justificacion                                                                                                           |
|-------------------------------------------------------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Tipos posibles para variables                         | <b>Diseño del lenguaje</b>           | TypeScript define que tipos existen en su especificacion                                                                |
| Tipo de <code>count</code><br>( <code>number</code> ) | <b>Compilacion</b>                   | El compilador vincula el identificador <code>count</code> con el tipo <code>number</code> al procesar la declaracion    |
| Rango de valores de <code>number</code>               | <b>Implementacion del compilador</b> | IEEE 754 float64: $\pm 5 \times 10^{-324}$ a $\pm 1.8 \times 10^{308}$ — decision de los implementadores del compilador |
| Significado del operador <code>+</code>               | <b>Compilacion</b>                   | Se resuelve la sobrecarga: <code>number + number =</code> suma numerica                                                 |
| Representacion del literal <code>5</code>             | <b>Implementacion del compilador</b> | El compilador decide como representar el literal <code>5</code> en bytecode                                             |
| Valor de <code>count</code>                           | <b>Ejecucion (runtime)</b>           | Solo cuando la linea se ejecuta, <code>count</code> toma el valor calculado                                             |

**Paso 3 — Conclusion:** Dos lineas de codigo activan cinco momentos de binding simultaneamente. El tipo se vincula en compilacion (estatico); el valor se vincula en ejecucion (dinamico). El rango del tipo es decision de implementacion, no del programador ni del disenador del lenguaje.

**Ver filmina:** [F-10]

## Ejemplo 2: Resolucion de ambito estatico paso a paso

**Enunciado:** Dado el siguiente codigo, determinar que imprime cada `console.log` y por que.

```

let x = 10; // ambito: modulo (nivel 0)

function externa() {
  let y = 20; // ambito: funcion externa (nivel 1)

  function interna() {
    let z = 30; // ambito: funcion interna (nivel 2)

    console.log(x); // (A)
    console.log(y); // (B)
    console.log(z); // (C)
  }

  // console.log(z); // (D)
}

// console.log(y); // (E)

```

## Resolucion paso a paso:

### (A) `console.log(x)` dentro de `interna()` :

1. Buscar `x` en nivel 2 (bloque de `interna`): No se encuentra.
2. Subir al nivel 1 (bloque de `externa`): No se encuentra.
3. Subir al nivel 0 (modulo): Se encuentra `x = 10`.
4. Resultado: imprime `10`.

### (B) `console.log(y)` dentro de `interna()` :

1. Buscar `y` en nivel 2 (bloque de `interna`): No se encuentra.
2. Subir al nivel 1 (bloque de `externa`): Se encuentra `y = 20`.
3. Resultado: imprime `20`.

### (C) `console.log(z)` dentro de `interna()` :

1. Buscar `z` en nivel 2 (bloque de `interna`): Se encuentra `z = 30`.
2. Resultado: imprime `30`. No necesita subir mas.

### (D) `console.log(z)` dentro de `externa()` (comentado):

1. Buscar `z` en nivel 1 (bloque de `externa`): No se encuentra.
2. Subir al nivel 0 (modulo): No se encuentra.
3. Resultado: **error de compilacion** — `z` no existe en nivel 1 ni arriba.

### (E) `console.log(y)` en el modulo (comentado):

1. Buscar `y` en nivel 0 (modulo): No se encuentra.
2. No hay mas niveles hacia afuera.
3. Resultado: **error de compilacion** — `y` no existe en nivel 0.

**Conclusion:** El algoritmo de resolucion de ambito estatico sube por la cadena lexica (la estructura del codigo fuente) hasta encontrar el nombre o fallar. Todo se resuelve en compilacion — el

compilador ya sabe que variable es cada nombre antes de ejecutar.

**Ver filmina:** [F-29]

---

### Ejemplo 3: Deteccion de los 3 patrones de errores de IA

**Enunciado:** Identificar cuantos errores o problemas hay en el siguiente fragmento y a que patron corresponde cada uno.

```
let limite = 100;
var total = 0;

function procesar(items: number[]) {
  if (items.length > 0) {
    var resultado = items[0] * 2;
  }
  for (const item of items) {

    const limite = item;
    total += limite;
  }
  console.log(resultado);
  return items.filter(x => x < limite);
}
```

#### Resolucion paso a paso:

##### Error 1 — Patron 1 ( var hoisting):

- Linea: `var resultado = items[0] * 2;`
- Problema: `var resultado` se eleva al inicio de `procesar()`. Fuera del `if`, `console.log(resultado)` puede imprimir `undefined` si `items.length === 0` — sin lanzar error.
- Solucion: usar `let resultado` en lugar de `var`. Fuera del bloque del `if`, daria `ReferenceError` explicito.

##### Error 2 — Patron 2 (variable global silenciosa):

- Linea: `var total = 0;` (en el modulo) y `total += limite;` (dentro de `procesar`).
- Problema: `total` es una variable global mutable que acumula estado entre llamadas. La firma de `procesar` no refleja esta dependencia. El resultado de la funcion cambia segun cuantas veces se llamo antes.
- Solucion: pasar `total` como parametro explicito.

##### Error 3 — Patron 3 (shadowing inesperado):

- Línea: `const limite = item;` dentro del `for`.
- Problema: `limite` local oculta al `limite = 100` del módulo. El `filter` final usa el `limite` del módulo (100) porque el `limite` del `for` ya salió de scope al terminar el bucle. Pero dentro del `for`, `total += limite` usa el `limite` local (`item`), no el 100. Esto es confuso y propenso a bugs.
- Solución: usar nombres diferentes para variables locales y exteriores (ej. `const valorItem = item`).

**Conclusion:** Tres patrones distintos en un solo fragmento. `var resultado` → hoisting (Patrón 1). `var total` global → estado oculto (Patrón 2). `const limite` → shadowing (Patrón 3). TypeScript con strict mode detecta algunos de estos automáticamente, pero el shadowing requiere ESLint `no-shadow`.

**Ver filmina:** [F-45]

## 6. Puntos clave (cheat-sheet)

### Los 6 atributos de una variable

| Atributo       | Notacion | Que es                                          |
|----------------|----------|-------------------------------------------------|
| Nombre         | —        | Identificador simbolico (puede no existir)      |
| Direccion      | L-value  | Celda(s) de memoria                             |
| Tipo           | —        | Rango de valores + operaciones + representacion |
| Valor          | R-value  | Contenido codificado                            |
| Tiempo de vida | lifetime | Periodo vinculado a una direccion               |
| Ambito         | scope    | Rango donde el nombre es visible                |

### L-value vs. R-value

- **L-value** = direccion (destino de escritura). Aparece a la izquierda del `=`.
- **R-value** = contenido (fuente de lectura). Aparece a la derecha del `=`.
- Una constante tiene L-value pero el lenguaje prohíbe reasignarla.

## Los 6 tiempos de binding

1. **Diseño del lenguaje** — significados de símbolos ( \* = multiplicación)
2. **Implementación del compilador** — rango de tipos primitivos (IEEE 754)
3. **Compilación** (estático) — variable → tipo; operador → semántica
4. **Linking** — llamada externa → código real
5. **Carga del módulo** — variables globales → celdas
6. **Ejecución** (dinámico) — variable → valor concreto

## Las 4 categorías de variables

| Categoría         | Zona              | Recursion? | Ejemplo                                          |
|-------------------|-------------------|------------|--------------------------------------------------|
| 1. Estáticas      | Segmento estático | No         | <code>const VERSION = "1.0.0"</code>             |
| 2. Stack-dynamic  | Pila de llamadas  | Si         | <code>let resultado = 0</code> dentro de función |
| 3. Heap explícita | Heap              | Si         | <code>new Nodo(42)</code>                        |
| 4. Heap implícita | Heap              | Si         | <code>x = 42</code> en Python (todo cambia)      |

## Las 2 dimensiones del binding de tipos (ortogonales)

|                 | Fuerte                           | Debil           |
|-----------------|----------------------------------|-----------------|
| <b>Estático</b> | Haskell, Rust, TypeScript strict | C, C++          |
| <b>Dinámico</b> | Python                           | JavaScript, PHP |

## Ámbito estático vs. dinámico

|                        | Estático (lexico)     | Dinámico           |
|------------------------|-----------------------|--------------------|
| <b>Resolución</b>      | Compilación           | Ejecución          |
| <b>Determinado por</b> | Estructura del código | Cadena de llamadas |
| <b>Verificable</b>     | Si                    | No                 |
| <b>Legible</b>         | Alta                  | Baja               |

## Los 3 patrones de errores de IA

1. **var hoisting** — `var` se eleva al inicio de la función → `undefined` silencioso
2. **Variable global silenciosa** — efecto secundario oculto no declarado en la firma
3. **Shadowing inesperado** — nombre local oculta al exterior → referencia equivocada

## Tres ideas para llevarse

1. **La variable es una 5-tupla** — nombre, direccion, tipo, L-value, R-value — cada atributo tiene su propio binding y su propio momento.
2. **Binding ocurre en 6 momentos** — cuanto antes ocurre, mas eficiente y mas verificable; cuanto mas tarde, mas flexible pero mas riesgoso.
3. **El ambito estatico es predecible** — se resuelve en compilacion: cualquier error de visibilidad aparece antes de ejecutar el programa.

---

## 7. Autoevaluacion

---

Las siguientes 10 preguntas cubren distintos niveles de Bloom. Intenta responderlas sin consultar la guia. Las respuestas estan al final, dentro de un bloque desplegable.

### Preguntas

**P1 (Recordar)** — Cuales son los 6 atributos de una variable segun Sebesta? Lista los 6 y su notacion formal.

**P2 (Recordar)** — Cuales son las 4 categorias de variables segun su binding de almacenamiento? Para cada una, indica si permite recursion.

**P3 (Comprender)** — En la sentencia `x = y + 1`, cual variable actua como L-value y cual como R-value? Explica por que.

**P4 (Comprender)** — Por que las variables estaticas (Categoria 1) no permiten recursion efectiva? Explica el mecanismo.

**P5 (Aplicar)** — Dado el siguiente codigo TypeScript, aplica el algoritmo de resolucion de ambito estatico para determinar que imprime `console.log(x)` dentro de `interna()`:

```
let x = 5;
function externa() {
  let x = 10;
  function interna() {
    console.log(x); // que imprime?
  }
  interna();
}
externa();
```

**P6 (Aplicar)** — Clasifica cada uno de los siguientes bindings por su momento (diseno, implementacion, compilacion, linkeo, carga, ejecucion):



- (a) El significado de `*` como multiplicacion
- (b) El valor de la variable `contador` despues de `contador = 42`
- © El tipo de `let n: number`
- (d) El rango de valores de `number` (IEEE 754 float64)
- (e) La direccion de memoria de una variable global al importar el modulo

**P7 (Analizar)** — Compara estos dos lenguajes y completa la tabla:

|                                     | TypeScript strict | Python |
|-------------------------------------|-------------------|--------|
| Binding de tipo (estatico/dinamico) | ?                 | ?      |
| Fuerza del tipado (fuerte/debil)    | ?                 | ?      |
| Cuando se detecta un error de tipo? | ?                 | ?      |

**P8 (Analizar)** — En el siguiente codigo, identifica si hay shadowing y explica que imprime cada `console.log` :

```
let x = 10;
function procesar(items: number[]) {
  for (const item of items) {
    const x = item * 2;
    console.log(x); // (A)
  }
  console.log(x); // (B)
}
procesar([5, 10, 15]);
```

**P9 (Evaluar)** — El siguiente codigo fue generado por una IA. Identifica los tres patrones de error de scope vistos en clase y explica por que cada uno es problematico:

```
let total = 0;
var resultado;
function procesar(items: number[]) {
  const limite = 100;
  for (const item of items) {
    const limite = item;
    total += limite;
  }
  var resultado = items[0];
  return total;
}
```

**P10 (Crear)** — Escribe un prompt seguro para pedirle a una IA que genere una funcion TypeScript que sume los elementos de un array, siguiendo las buenas practicas de scope vistas en clase. El prompt debe prevenir los tres patrones de error.

► **Respuestas**

## 8. Glosario

| Termino                       | Definicion                                                                                                                                                                                                             |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Variable</b>               | Abstraccion de una celda de memoria; formalmente una 5-tupla <code>&lt;nombre, direccion, tipo, L-valor, R-valor&gt;</code> con tiempo de vida y ambito como atributos de contexto [Sebesta, §5.3, Cap. 5]             |
| <b>L-value</b>                | (Left value) Direccion de memoria asociada a una variable; representa el destino de escritura en una asignacion. Aparece a la izquierda del <code>=</code> [Gabbrielli & Martini, §8.4, Cap. 8]                        |
| <b>R-value</b>                | (Right value) Contenido codificado almacenado en la celda, interpretado segun el tipo; representa la fuente de lectura en una asignacion. Aparece a la derecha del <code>=</code> [Sebesta, §5.3.2, Cap. 5]            |
| <b>Binding</b>                | (Vinculacion) Asociacion entre una entidad del programa y uno de sus atributos, establecida en un momento determinado [Sebesta, §5.4, Cap. 5]                                                                          |
| <b>Binding time</b>           | (Tiempo de binding) El momento en que ocurre una vinculacion. Puede ser: diseno, implementacion, compilacion, linkeo, carga o ejecucion [Sebesta, §5.4, Cap. 5]                                                        |
| <b>Binding estatico</b>       | Binding que ocurre antes de que comience la ejecucion y permanece sin cambios durante toda la ejecucion del programa [Sebesta, §5.4, Cap. 5]                                                                           |
| <b>Binding dinamico</b>       | Binding que ocurre durante la ejecucion o puede cambiar en el curso de la ejecucion del programa [Sebesta, §5.4, Cap. 5]                                                                                               |
| <b>Type inference</b>         | (Inferencia de tipos) Proceso por el cual el compilador deduce el tipo de una expresion sin declaracion explicita del programador. Ocurre en compilacion — no es binding dinamico [Gabbrielli & Martini, §8.8, Cap. 8] |
| <b>Coercion</b>               | Conversion de tipo insertada automaticamente por el compilador o runtime cuando detecta compatibilidad entre tipos. En tipado debil, ocurre silenciosamente [Gabbrielli & Martini, §8.3, Cap. 8]                       |
| <b>Tipado fuerte</b>          | Sistema donde cada operacion verifica compatibilidad de tipos y no hay conversiones implicitas inseguras. Los errores de tipo son explicitos [Gabbrielli & Martini, §8.3, Cap. 8]                                      |
| <b>Tipado debil</b>           | Sistema que permite conversiones implicitas arbitrarias. Los errores de tipo producen resultados silenciosos posiblemente incorrectos [Gabbrielli & Martini, §8.3, Cap. 8]                                             |
| <b>Variable estatica</b>      | (Categoria 1) Variable vinculada a una direccion antes de la ejecucion; permanece hasta el fin del programa. No permite recursion [Sebesta, §5.4.3.1, Cap. 5]                                                          |
| <b>Variable stack-dynamic</b> | (Categoria 2) Variable cuyo binding de almacenamiento se crea al elaborar su declaracion (en cada llamada al subprograma); se libera al retornar. Permite recursion [Sebesta, §5.4.3.2, Cap. 5]                        |

| Termino                                | Definicion                                                                                                                                                                                                                    |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Variable heap-dynamic explicita</b> | (Categoria 3) Celda de memoria sin nombre asignada y liberada por instrucciones explicitas del programador ( <code>new</code> , <code>Box : new</code> ). Solo accesible via puntero o referencia [Sebesta, §5.4.3.3, Cap. 5] |
| <b>Variable heap-dynamic implicita</b> | (Categoria 4) Variable cuyos todos los atributos se vinculan cada vez que se le asigna un valor. Ejemplo canonico: Python [Sebesta, §5.4.3.4, Cap. 5]                                                                         |
| <b>Scope (ambito)</b>                  | Rango de instrucciones del programa donde el nombre de una variable es visible y puede ser referenciado [Sebesta, §5.5, Cap. 5]                                                                                               |
| <b>Ambito estatico (lexico)</b>        | Regla de ambito donde la visibilidad se determina por la estructura lexica del codigo fuente, en tiempo de compilacion. Introducido por ALGOL 60 [Sebesta, §5.5, Cap. 5]                                                      |
| <b>Ambito dinamico</b>                 | Regla de ambito donde la visibilidad se determina por la cadena de llamadas activas en runtime. La asociacion valida es la mas reciente creada para el nombre que sigue activa [Gabbrielli & Martini, §4.3, Cap. 4]           |
| <b>Shadowing</b>                       | Ocurre cuando una declaracion en un bloque interno usa el mismo nombre que una variable del bloque exterior, ocultandola. La variable exterior cae en un "scope hole"                                                         |
| <b>Scope hole</b>                      | (Agujero de ambito) Region del codigo donde una variable exterior existe en memoria pero su nombre no es visible porque fue sombreado por una variable interior con el mismo nombre                                           |
| <b>Referencing environment</b>         | (Entorno de referencia) Coleccion de todos los identificadores visibles en una sentencia dada. En ambito estatico: variables locales + variables de ambitos ancestros visibles [Sebesta, §5.5, Cap. 5]                        |
| <b>Constant</b>                        | (Constante) Identificador cuyo valor se fija en un momento determinado y el lenguaje prohíbe modificarlo despues. Representa un binding de R-value inmutable; el L-value sigue existiendo [Sebesta, §5.4.3, Cap. 5]           |
| <b>Initialization</b>                  | (Inicializacion) Binding de una variable a un valor en el momento en que se vincula al almacenamiento [Sebesta, §5.4.3, Cap. 5]                                                                                               |
| <b>Elaboration</b>                     | (Elaboracion) Proceso de asignacion de memoria y vinculacion indicado por la declaracion, que ocurre cuando la ejecucion alcanza el codigo al que esta asociada la declaracion [Sebesta, §5.4.3.2, Cap. 5]                    |
| <b>Allocation</b>                      | (Asignacion de memoria) Proceso de tomar una celda del conjunto de memoria disponible para vincularla a una variable [Sebesta, §5.4.3, Cap. 5]                                                                                |
| <b>Deallocation</b>                    | (Liberacion de memoria) Proceso de devolver al conjunto de memoria disponible una celda que ha dejado de estar vinculada a una variable [Sebesta, §5.4.3, Cap. 5]                                                             |
| <b>Hoisting</b>                        | (Elevacion) Mecanismo de JavaScript/TypeScript donde las declaraciones <code>var</code> se mueven al inicio del scope de funcion. La declaracion sube; la inicializacion permanece en su lugar                                |

| Termino                         | Definicion                                                                                                                                                                                |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Temporal Dead Zone (TDZ)</b> | Region del scope donde una variable <code>let</code> / <code>const</code> existe pero no puede usarse hasta que se alcanza su declaracion. Usarla antes lanza <code>ReferenceError</code> |
| <b>Activation record</b>        | (Registro de activacion / frame) Estructura en la pila de llamadas que contiene las variables locales y parametros de una invocacion de subprograma. Cada llamada crea su propio frame    |

## 9. Referencias y lecturas recomendadas

### Fuentes primarias (verificadas en ChromaDB)

1. **Sebesta, R. W.** (2019). *Concepts of Programming Languages* (12th ed.). Pearson.
  - Cap. 5, §5.3–5.8 (pp. 221–258): atributos de variables, binding, tiempos de binding, binding de tipos, storage bindings y lifetime, scope, entorno de referencia, constantes, inicializacion.
  - Cap. 5, §5.4.3.1–5.4.3.4: las 4 categorias de variables (estaticas, stack-dynamic, heap explicita, heap implicita).
  - Cap. 5, §5.5.4–5.5.5: problemas del ambito dinamico, evaluacion del ambito estatico.
  - Cap. 9, §9.3: activation records y stack frames (referenciado para recursion).
2. **Gabbrielli, M. & Martini, S.** (2023). *Programming Languages: Principles and Paradigms* (2nd ed.). Springer.
  - Cap. 4, §4.3 (pp. 083–105): nombres, ambito estatico y dinamico, definicion formal de ambito dinamico, fases de linking y loading.
  - Cap. 5 (pp. 106–135): implementacion de reglas de ambito, static chain y display.
  - Cap. 8, §8.3–8.8 (pp. 136–282): L-value/R-value, tipado estatico/dinamico, fuerte/debil, coerciones, inferencia de tipos.
  - Cap. 14: ownership en Rust como alternativa al GC.
3. **Louden, K. C. & Lambert, K. A.** (2012). *Programming Languages: Principles and Practices* (3rd ed.). Course Technology.
  - Cap. 7, §7.5 (pp. 210–330): subcategorias del binding estatico (translation, link, load time) y dinamico (entrada/salida de procedimiento, asignacion).

### Documentacion oficial

4. **TypeScript Handbook.** Variable Declarations, Strict Mode.  
<https://www.typescriptlang.org/docs/>
  - `let` vs `const` vs `var`, strict mode, analisis de flujo de tipos, inferencia bidireccional.

## Lecturas recomendadas para profundizar (Tema 09.2)

5. **Sebesta, R. W.** (2019). Cap. 5, §5.6 (alias), §5.7 (referencing environments en closures). Para Tema 09.2.
6. **Gabbrielli, M. & Martini, S.** (2023). Cap. 4, §4.3 (closures y ambito), Cap. 14 (garbage collection y ownership). Para Tema 09.2.

---

 **Guia de estudio generada por Dra. Sofia — Study Guide Writer (EDU) Tema 09.1: Variables, Binding y Ambito** — 1 clase × 120 min **Lenguaje principal:** TypeScript **Fuentes:** Sebesta Cap. 5 + Gabbrielli & Martini Caps. 4/5/8/14 + Louden Cap. 7 **Baseline:** `clase_dada.txt` (filminas reales dadas en clase 2026-06-28) **Citas verificadas en ChromaDB** — knowledge base con 3 libros ingestados **Scope:** estricto a `diseño.md` — sin extender a temas de 09.2 (alias, closures, GC)